

2520080012

OSSP SKILL 6

Part B:

```
GNU nano 7.2 server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define CLIENT_FIFO "client_fifo"
#define SERVER_FIFO "server_fifo"

int main()
{
    char message[100];

    // Create FIFOs
    mkfifo(CLIENT_FIFO, 0666);
    mkfifo(SERVER_FIFO, 0666);

    printf("Server started...\n");

    while (1)
    {
        // Open client FIFO for reading
        int client_fd = open(CLIENT_FIFO, O_RDONLY);

        // Read message from client
        read(client_fd, message, sizeof(message));
        close(client_fd);

        printf("Client: %s\n", message);

        // Exit condition
        if (strcmp(message, "exit") == 0)
            break;

        // Send response
        int server_fd = open(SERVER_FIFO, O_WRONLY);

        char response[100];
        snprintf(response, sizeof(response),
                "Server received: %s", message);

        write(server_fd, response, strlen(response) + 1);
        close(server_fd);
    }

    unlink(CLIENT_FIFO);
    unlink(SERVER_FIFO);

    printf("Server stopped.\n");

    return 0;
}

root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ mkdir -p ~/skill6
alurusaranya@LAPTOP-0PA61QEN:~$ cd ~/skill6
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ mkfifo myfifo
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ls -l myfifo
prw-rw-r-- 1 alurusaranya alurusaranya 0 Sep 21 02:57 myfifo
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ pwd
/home/alurusaranya/skill6
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ nano server.c
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ gcc server.c -o server
server.c: In function 'main':
server.c:41:36: warning: '%s' directive output may be truncated writing up to 99 bytes into a region of size 83 [-Wformat-truncation=]
   41 |         "Server received: %s", message);
      |                                     ^
server.c:40:9: note: 'snprintf' output between 18 and 117 bytes into a destination of size 100
   40 |         snprintf(response, sizeof(response),
      |         ^~~~~~
   41 |         "Server received: %s", message);

alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ls
myfifo server server.c
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ nano client.c
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ gcc client.c -o client
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ls
client client.c myfifo server server.c
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ cd ~/skill6
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ./server
Server started...
Client: hello
Client: How are you?
Client: exit
Server stopped.
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ |
```

2520080012

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define CLIENT_FIFO "client_fifo"
#define SERVER_FIFO "server_fifo"

int main()
{
    char message[100];
    char response[100];

    printf("Client started...\n");

    while (1)
    {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        // Remove newline
        message[strcspn(message, "\n")] = '\0';

        // Send message to server
        int client_fd = open(CLIENT_FIFO, O_WRONLY);

        write(client_fd, message, strlen(message) + 1);
        close(client_fd);

        // Exit
        if (strcmp(message, "exit") == 0)
            break;

        // Receive response
        int server_fd = open(SERVER_FIFO, O_RDONLY);

        read(server_fd, response, sizeof(response));
        close(server_fd);

        printf("%s\n", response);
    }

    return 0;
}
```

TERMINAL 2

```
root@LAPTOP-0PA61QEN:~# ls /home
alurusaranya
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ cd ~/skill6
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ./client
Client started...
Enter message: hello
Server received: hello
Enter message: How are you?
Server received: How are you?
Enter message: exit
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ |
```

OTHER TERMINAL

```

root@LAPTOP-0PA61QEN:~# ls /home
alurusaranya
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ cd ~/skill6
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ls -l
total 48
-rwxrwxr-x 1 alurusaranya alurusaranya 16432 Sep 21 03:37 client
-rw-rw-r-- 1 alurusaranya alurusaranya 934 Sep 21 03:37 client.c
prw-rw-r-- 1 alurusaranya alurusaranya 0 Sep 21 02:57 myfifo
-rwxrwxr-x 1 alurusaranya alurusaranya 16424 Sep 21 03:37 server
-rw-rw-r-- 1 alurusaranya alurusaranya 1103 Sep 21 03:37 server.c

```

Part A — Named Pipes (FIFO) Theory

A **named pipe**, also called a **FIFO (First In, First Out)**, is a special type of file used for communication between processes. It provides a mechanism through which one process can send data to another process.

Unlike an ordinary pipe, a named pipe has a name in the file system and can be accessed by unrelated processes. One process can write data into the FIFO while another process reads the data from it.

In Linux, the `mkfifo()` system call is used to create a named pipe. The `open()`, `read()`, and `write()` system calls are used to access and transfer data through the FIFO.

In client-server communication using FIFOs, the **client** sends a message through one FIFO and the **server** reads the message and sends a response through another FIFO. This allows two processes to communicate and exchange information.

The FIFO follows the **First In, First Out** principle, meaning that data written first is read first. Named pipes provide a simple method of **inter-process communication (IPC)** in Linux.

PART-B

```

root@LAPTOP-0PA61QEN:~# ls /home
alurusaranya
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ cd ~/skill6
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ls
client client.c myfifo server server.c signal signal.c
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ nano signal.c
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ gcc signal.c -o signal
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ ./signal
Process ID: 917
Waiting for signals...

Received SIGUSR1

Received SIGINT

Received SIGTERM
Killed
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ |

```

```

GNU nano 7.2 signal.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int signal)
{
    if (signal == SIGINT)
    {
        printf("\nReceived SIGINT\n");
    }
    else if (signal == SIGTERM)
    {
        printf("\nReceived SIGTERM\n");
    }
    else if (signal == SIGUSR1)
    {
        printf("\nReceived SIGUSR1\n");
    }
}

int main()
{
    printf("Process ID: %d\n", getpid());

    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Waiting for signals...\n");

    while (1)
    {
        pause();
    }

    return 0;
}

```

```
root@LAPTOP-0PA61QEN:~# ls /home
alurusaranya
root@LAPTOP-0PA61QEN:~# su alurusaranya
alurusaranya@LAPTOP-0PA61QEN:/root$ cd ~
alurusaranya@LAPTOP-0PA61QEN:~$ cd ~/skill6
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ kill -SIGUSR1 917
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ kill -SIGINT 917
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ kill -SIGTERM 917
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ kill -9 917
alurusaranya@LAPTOP-0PA61QEN:~/skill6$ |
```

A signal is a software notification sent to a process to inform it that a particular event has occurred. Signals provide a way for processes and the operating system to communicate with each other.

In Linux, a process can receive different types of signals. The SIGINT signal is used to interrupt a process, SIGTERM is used to request termination of a process, and SIGUSR1 is a user-defined signal that can be used for application-specific purposes.

A signal handler is a function that is executed when a particular signal is received. The `signal()` function is used to associate a signal with its corresponding handler. The process ID (PID) identifies a process uniquely. The `kill` command can be used to send a signal to a process using its PID.

Signal handling allows a program to respond to events without continuously checking for them. This provides an effective mechanism for controlling and coordinating processes in a Linux operating system.